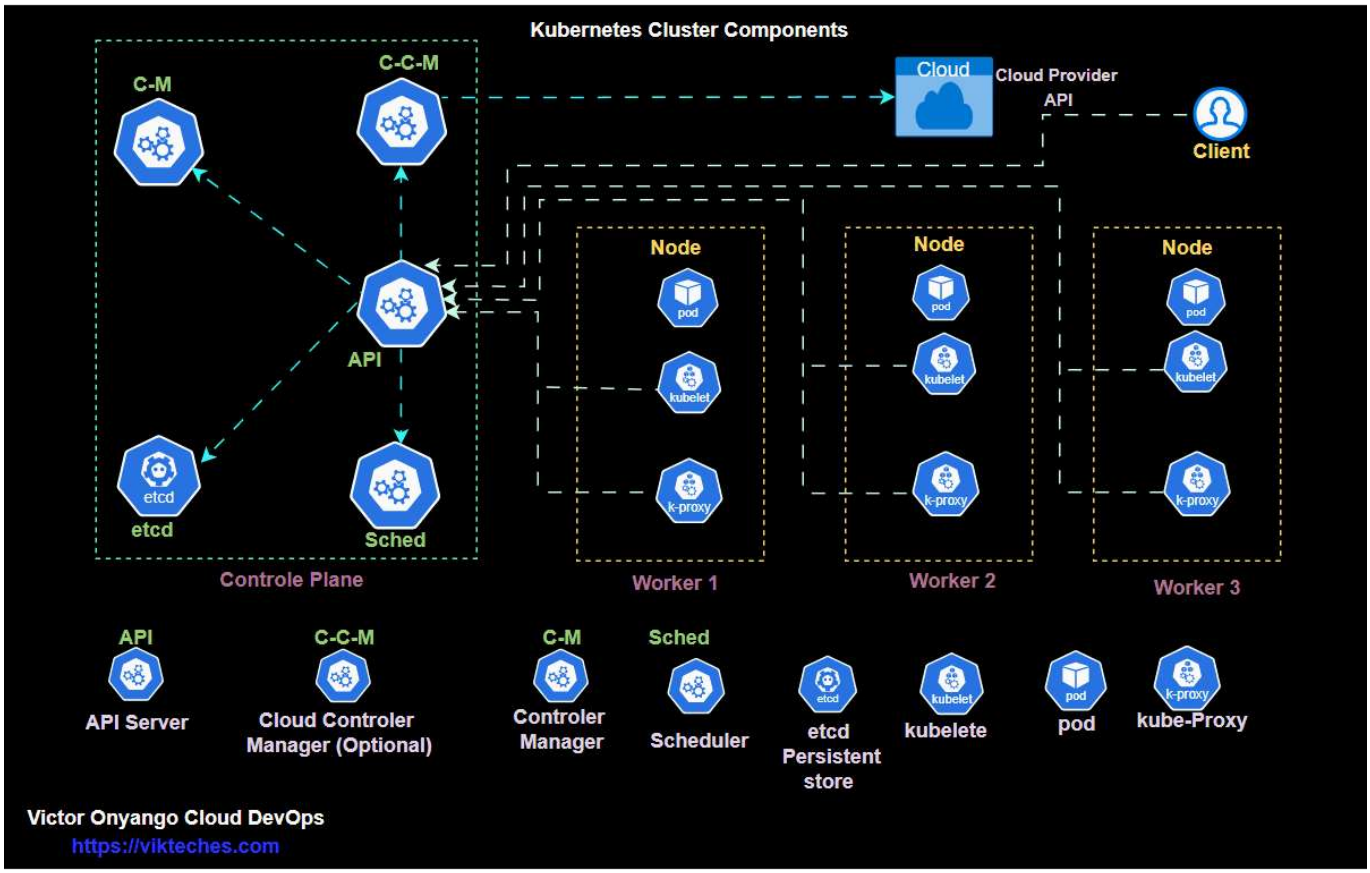
Open in app ↗

Medium Search Write 6

Kubernetes Architecture: A Deep Dive into the Core Components

Victor wasonga onyango Follow 5 min read · Oct 31, 2024

43 1



Kubernetes has transformed how we deploy, manage, and scale containerized applications, offering a robust orchestration framework for high availability, scalability, and efficiency. In this article, we'll explore

Kubernetes architecture and its main components, helping you gain a solid understanding of how Kubernetes works under the hood.

Overview of Kubernetes Architecture

Kubernetes uses a **cluster-based architecture** in which components collaborate to manage and orchestrate containerized applications across a set of machines, known as nodes. The architecture consists of two main layers:

1. **Control Plane**: Manages the overall cluster and its resources.
2. **Node (Worker) Plane**: Runs the application workloads.

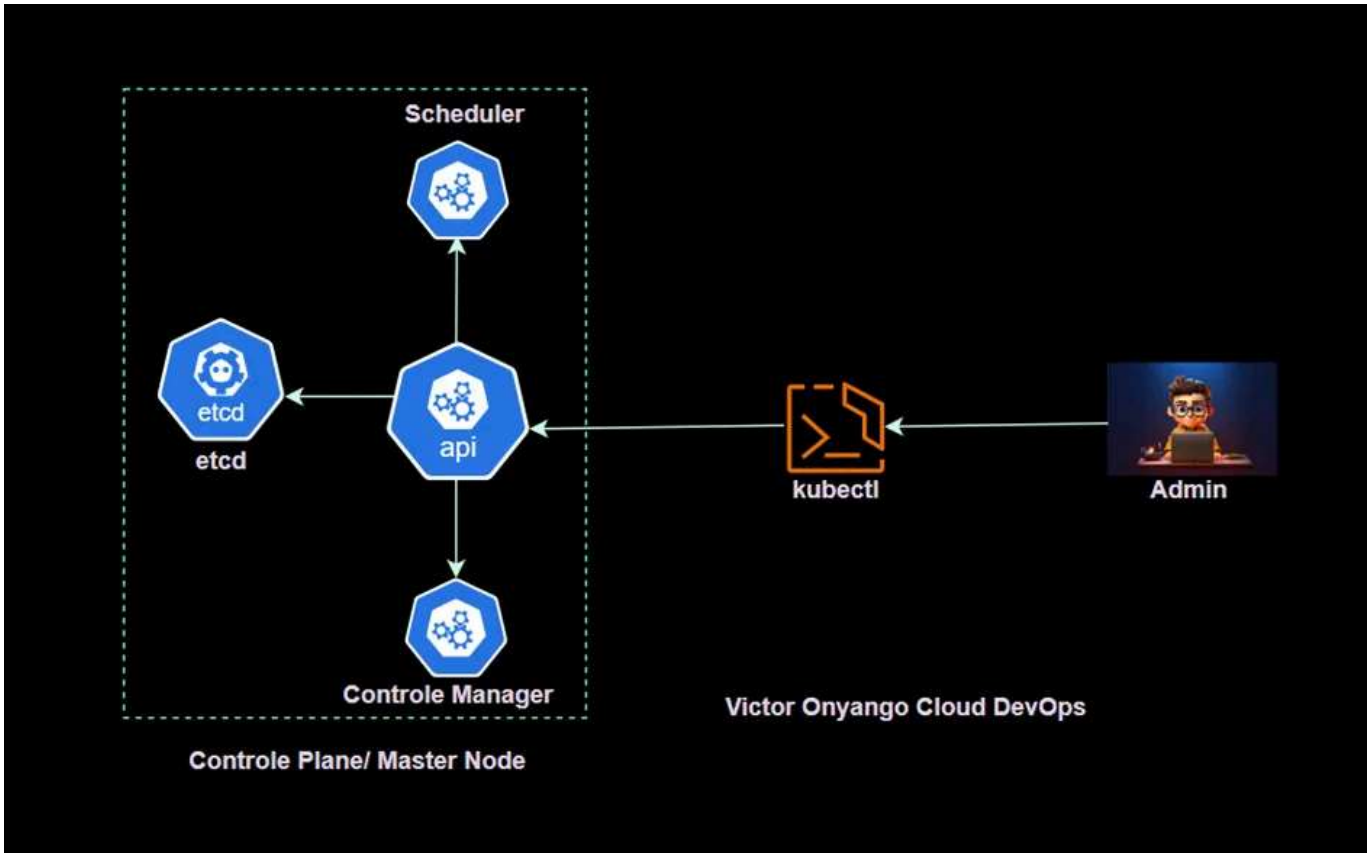
Together, these components provide a resilient and scalable environment for deploying applications, managing workloads, and ensuring they run as expected.

Core Components of Kubernetes Architecture

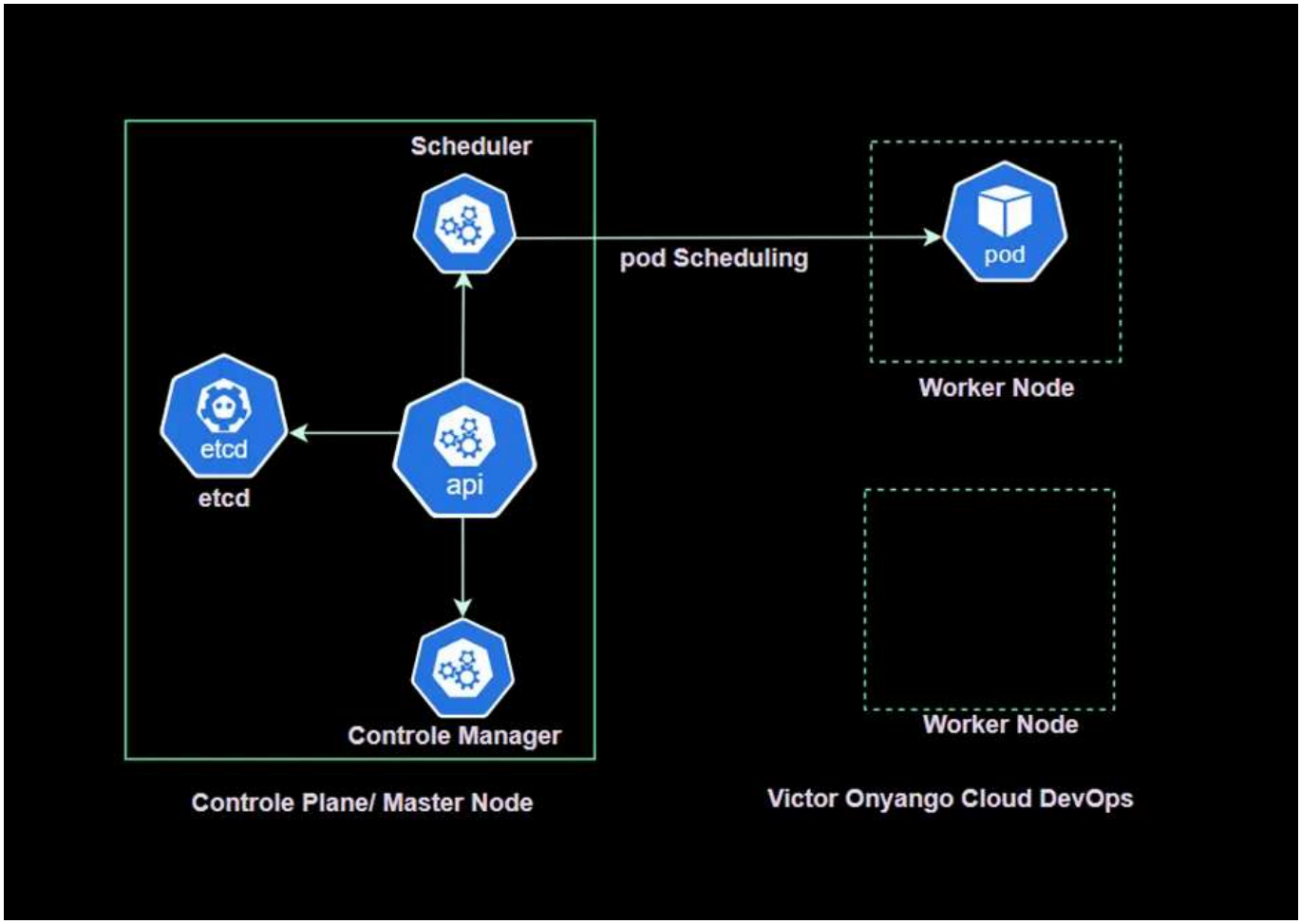
1. Control Plane

The control plane serves as the command center of the Kubernetes cluster, handling all aspects of cluster management. It includes several crucial components:

- **API Server**: The API server is the front-end for the Kubernetes control plane, exposing the Kubernetes API for both internal and external communication with the cluster. It is the main contact point for developers, administrators, and services interacting with Kubernetes, making it essential to all cluster management activities.



- **Etcd**: Etcd is a distributed key-value store that Kubernetes uses to store configuration data and state information about the cluster. It ensures any configuration or state changes are recorded and accessible to control plane components, enabling high availability and cluster consistency.
- **Controller Manager**: The controller manager runs various controllers that monitor the cluster's desired state and make changes as needed. For example, the replication controller ensures the correct number of pod replicas are running, while the node controller checks node health and availability.
- **Scheduler**: The scheduler assigns pods to nodes based on resource requirements, workload distribution, and availability. It considers factors like resource limits, affinity/anti-affinity rules, and workload distribution to deploy applications optimally within the cluster.



2. Node (Worker) Plane

Worker nodes are responsible for running containerized applications and workloads. Each node in a Kubernetes cluster includes the following components:

- **Kubelet:** Kubelet is an agent that runs on each node, receiving information about scheduled pods from the API server. It ensures containers described in pod manifests are running and remain in the desired state.
- **Kube-proxy:** Kube-proxy manages network communication within the Kubernetes cluster. It enables smooth communication between services within the cluster by routing traffic to the appropriate pod based on defined services and network policies.

- **Container Runtime**: The container runtime (e.g., Docker, containerd) is the software responsible for running and managing containers. It pulls container images, starts/stops containers, and manages networking and storage for containers.
- **Container Storage Interfaces (CSI)**: For persistent data, Kubernetes uses Container Storage Interfaces, allowing various storage providers to offer persistent storage volumes for applications within the cluster.

Pods

Pods are the foundational building blocks of Kubernetes, grouping one or more containers that share network and storage contexts.

1. **Container Co-location**: Containers within a pod share the same network namespace, allowing them to communicate easily and share an IP address and port space.
2. **Shared Storage and Volumes**: Containers within a pod can share storage volumes, simplifying data exchange between them.
3. **Single Unit of Deployment**: Kubernetes deploys pods as the smallest unit. Scaling or managing an application involves working with pod replicas.
4. **Init Containers**: Pods can include init containers, which run initialization tasks before the main application containers start.

Controllers

Controllers ensure resources in the cluster match their desired states, automating tasks like scaling, self-healing, and application management.

1. **Replication Controller**: Manages the specified number of identical pod replicas, creating new pods if any fail or are deleted.
2. **Deployment**: Provides declarative updates for applications, including rolling updates that gradually replace old pods with new ones to avoid disruption.
3. **StatefulSet**: Manages stateful applications requiring unique identities and persistent storage.
4. **DaemonSet**: Ensures a specific pod runs on every node in the cluster, useful for deploying monitoring agents or networking components.
5. **Horizontal Pod Autoscaler (HPA)**: Adjusts the number of pod replicas based on CPU or custom metrics, allowing applications to adapt to changing demand.
6. **Vertical Pod Autoscaler (VPA)**: Adjusts resource requests and limits based on usage, optimizing resource allocation for efficiency.

Services

Kubernetes services enable communication and load balancing between different sets of pods, making applications discoverable and resilient.

1. **Service Types**:

- **ClusterIP**: Exposes the service within the cluster for internal communication.
- **NodePort**: Exposes the service on a static port on each node, suitable for development/testing.

- **LoadBalancer:** Creates an external load balancer for internet or external network access.
- **ExternalName:** Maps the service to a DNS name, enabling access to external services.

1. **Selectors and Labels:** Services use selectors to identify target pods based on labels, defining which pods the service should include.
2. **Load Balancing:** Services balance traffic across multiple pods, ensuring no single pod becomes overwhelmed.

Volumes

Volumes provide persistent storage for containers within pods, enabling data to persist across restarts or rescheduling.

Types of Volumes:

- **EmptyDir:** Temporary storage deleted when the pod is removed.
- **HostPath:** Mounts a host filesystem file/directory into the container.
- **PersistentVolumeClaim (PVC):** Requests persistent storage based on a storage class.
- **ConfigMap and Secret Volumes:** Injects data or sensitive information as files within containers.
- **Network File System (NFS):** Uses network-attached storage.
- **CSI Volumes:** Enables storage providers to integrate volume plugins with Kubernetes.

ConfigMaps and Secrets

- **ConfigMaps**: Store non-sensitive configuration data, such as settings and environment variables, for applications.
- **Secrets**: Store sensitive information, like passwords and tokens, offering an additional layer of security.

Namespaces

Namespaces are used to organize and partition resources within a cluster, creating virtual clusters within a physical cluster for different teams, projects, or environments.

1. **Purpose**: Namespaces provide resource isolation, management, access control, and tenant separation.
2. **Default Namespace**: Kubernetes creates a default namespace for resources if none is specified.
3. **Access Control and Quotas**: Namespaces enable Role-Based Access Control (RBAC) and resource quotas, limiting usage within a namespace.

Ingress

Ingress manages external access to services in the cluster, providing routing for HTTP and HTTPS traffic without modifying application code.

Ingress Controller: Watches for changes to ingress resources, configuring infrastructure to implement defined routing rules.

Conclusion

Kubernetes architecture is a sophisticated, powerful system for orchestrating containerized applications at scale. By distributing

responsibilities across the control plane and worker nodes, Kubernetes ensures high availability, scalability, and efficient resource utilization, supporting resilient deployments across dynamic, distributed environments. Thanks for reading and stay tuned for more.

For more of my projects click here. and for more of my blog posts, yoy can as well visit my website

Written by Victor wasonga onyango

262 followers · 157 following

Follow

A dedicated Cloud DevOps Eng. with a passion for streamlining development and operations processes in the cloud. Am also passionate about sharing knowledge.

Responses (1)

Piyali Das

What are your thoughts?

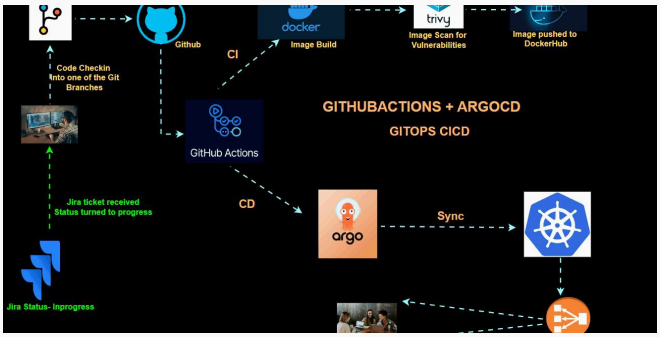
Agoba favour

Nov 5, 2024

This is wonderful.. kindly check out my article!!

Reply

More from Victor wasonga onyango

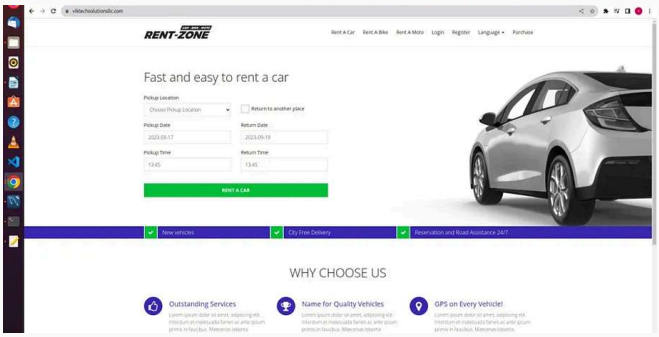


Victor wasonga onyango

GitOps CI/CD Pipeline with GitHub Actions & ArgoCD.

In today's fast-paced DevOps landscape, GitOps is emerging as the standard for secur...

May 19, 2025 31 2

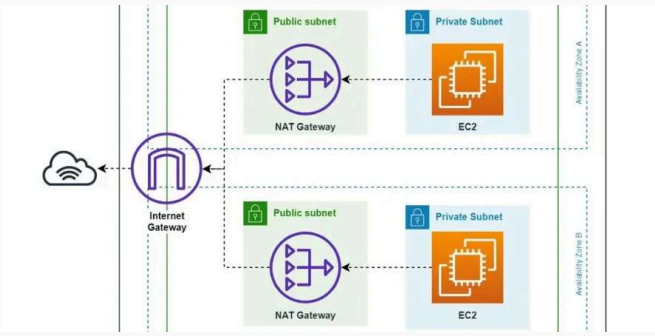


In Cloud Infra/DevOps Si... by Victor wasonga on...

Building a Dynamic Website on AWS: A Comprehensive Project...

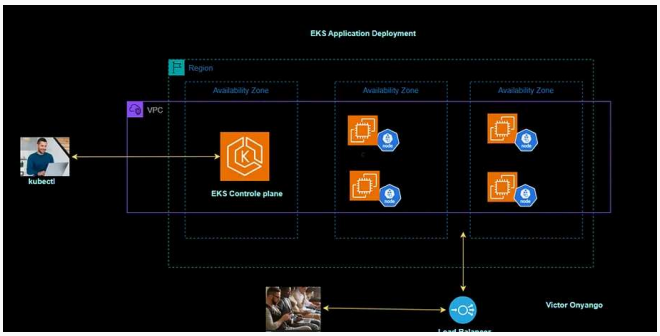
In this project, we will host this Rent-Zone Dynamic web application on AWS.

Oct 29, 2023 8 2



In Cloud Infra/DevOps Si... by Victor wasonga on...

Understanding AWS NAT Gateway: A Deep Dive into Network Addres...



In Cloud Infra/DevOps Si... by Victor wasonga on...

Simplified Guide to Deploying a Containerized Application on an...

To access the Internet, one public IP address is needed, but we can use a private IP addre...

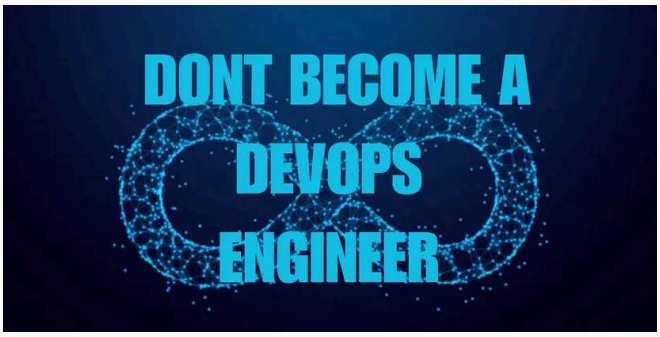
Nov 28, 2023 25 1

Kubernetes is an open-source container orchestration platform that automates the...

Mar 30, 2025

See all from Victor wasonga onyango

Recommended from Medium

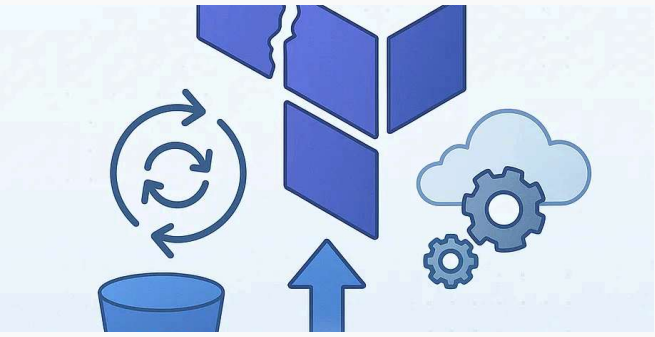


Dhanush N

Don’t Become a DevOps Engineer in 2026!

Stop writing YAML. Stop babysitting pipelines. The game has fundamentally...

Mar 2 477 17



Bharath Kumar Reddy N.

How to Recover from a Deleted Terraform State File (A Real...

Every DevOps engineer has a story that starts with a mistake. For me, it was a quiet...

Oct 13, 2025 8

Tech Brand

The DevOps Interview Questions You Can't Google Fast Enough

Why Your Quick Searches Aren't Saving You in DevOps Interviews

Dec 22, 2025 8 1

In Devops & AI Hub by Neel Shah

Top 10 MCP Servers To Speed Up DevOps Processes

Top Model Context Protocol (MCP) servers can dramatically speed up DevOps by lettin...

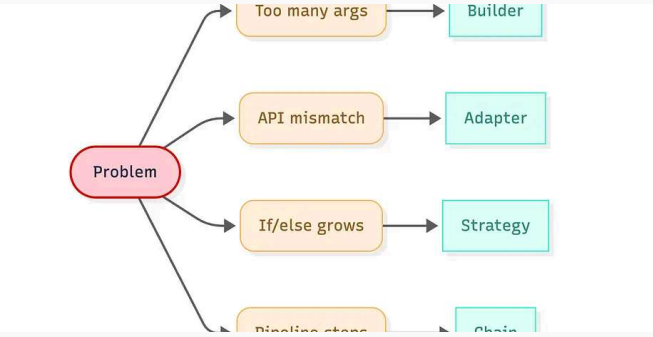
Jan 23 157 1

Bhavyansh

The CI/CD Setup That Cut Our Deployment Time from 40 Minute...

How we optimized our deployment pipeline and got back 32 minutes of our lives (per...

Oct 27, 2025 308 7



In Women in Technology by Alina Kovtun ✨

Stop Memorizing Design Patterns: Use This Decision Tree Instead

Choose design patterns based on pain points: apply the right pattern with minimal over-...

Jan 29 7.4K 74

See more recommendations